COM410 Programming in Practice (CRN 79082)

Assessment 1 (CW1)

Submission Deadline: Week 7 Lab Class

This assignment carries 40% of the overall mark for the module

This assignment addresses Learning Outcomes 3 and 4 of the module

---

IMPORTANT

1. This is an Open Book exercise. You may use module material from Blackboard and Internet searching is permitted. However, you may not use any Generative AI services such as ChatGPT etc.
2. You MUST use a lab machine for this exercise. All screen activity is being recorded and monitored.

TIME ALLOWED: 90 minutes

Introduction

The **Event Ticket Credit Wallet** system is used to manage digital ticket credits for event attendees. Each wallet stores the ticket holder's name and the number of remaining ticket credits. Ticket holders can add credits to their wallet (for example, when purchasing additional tickets) and use credits when attending events.

When a new wallet is created, the ticket holder may choose to enable a **bonus credit allowance** of two credits. If this option is enabled, the ticket holder is permitted to use credits even when no credits remain, provided the usage does not exceed the bonus credit limit.

Whenever credits are successfully used to attend an event, the system generates a **ticket code** for that event. If no ticket has been successfully issued, no ticket code exists.

Object Properties

You have been provided with a Java file **EventTicketWallet.java** that describes an EventTicketWallet object by the following properties:

Instant Variables

- **holderName**
  A String value to store the name of the ticket holder
- **remainingCredits**
  The current number of remaining ticket credits as an int value
- **hasBonusAllowance**
  A boolean value to record whether the ticket holder has a 2-credit bonus allowance enabled

- **lastTicketCode**
  A String value used to store the most recently issued ticket code.
  This value should be null if no ticket has been successfully issued.

Methods List

- **public EventTicketWallet(String newHolder, int openingCredits, boolean bonus)**
  Creates a new EventTicketWallet object and initialises the ticket holder's name, the opening number of ticket credits, and whether a 2-credit bonus allowance is enabled. The ticket code should initially be null.
- **public void addCredits(int credits)**
  Adds the specified number of credits to the wallet's remaining ticket credits.
  This method does not perform validation and simply increases the current balance by the amount provided.
- **public boolean useCredits(int credits)**
  Attempts to subtract the specified number of credits from the wallet.
  The method returns:
  - false if the request is invalid (exceeds available credits or bonus allowance limit)
  - true if the credits are successfully deducted

  The remaining credits are updated only when the operation succeeds.
  When the operation succeeds, the wallet must record a ticket code that can be retrieved later.
- **public int getRemainingCredits()**
  Returns the current number of remaining ticket credits in the wallet.
- **public String getLastTicketCode()**
  Returns the most recently issued ticket code.
  If no ticket has been successfully issued, this method must return null.
- **public String toString()**
  Returns a formatted string describing the wallet, including the ticket holder's name and remaining credits.

Testing

The application is tested in two ways:

(i) Using a suite of **JUnit tests** specified in the file EventTicketWalletTest.java. Initially, this file contains a single test method openingCredits10IsSet() that checks the constructor sets the opening credit balance correctly.

(ii) Using the custom test framework specified in the file **Event.java.** This file creates multiple EventTicketWallet objects and verifies correct operation of the addCredits() and useCredits() methods, as well as ticket code generation.

Tasks

Create a new Java Project in **IntelliJ** named **EventTicketWallet** and place the files EventTicketWallet.java and Event.java into the main source folder, and EventTicketWalletTest.java into the test folder. Run both the JUnit tests and the Event class to ensure that the project has been properly set up.

Now modify EventTicketWallet.java and EventTicketWalletTest.java as described below to improve the application's operation and provide more extensive coverage of testing. Note that **Tasks 7 and 9 require code to be added to the EventTicketWallet class**, while the others require **JUnit tests** to be added to the EventTicketWalletTest class.

1) Add a new JUnit test method **openingCredits20IsSet()** that passes if a new wallet in the name of "Homer" with no bonus allowance is created with 20 ticket credits.

2) Add a new JUnit test method addCredits7() that opens a new wallet for Homer with 10 ticket credits (no bonus), calls **addCredits(7)**, and checks that the remaining credits are now 17.

3) Add a new JUnit test method addZero() that checks that calling addCredits(0) leaves the credit balance unchanged.

4) Add a new JUnit test method **useCreditsReturnsTrue()** that creates a new wallet for Homer with an opening balance of 10 ticket credits and no bonus allowance, calls the useCredits() method with a valid value, and checks that it returns true.

5) Add a new JUnit test method **useCredits5()** that creates a new wallet for Homer with 20 ticket credits (no bonus), calls useCredits(5), and checks that the remaining credits are now 15.

6) Add a new JUnit test method **useBeyondBonusLimit()** that creates a new wallet for Homer with 10 ticket credits and the bonus allowance enabled. Check that calling useCredits(13) returns false and that the remaining credits are unchanged.

7) Add code to the **useCredits()** method to enforce limits as follows:

(i) If there is no bonus allowance and the requested credits exceed the current remaining credits, return false and leave the credits unchanged.

(ii) If the bonus allowance is enabled and the requested credits would take the wallet below the 2-credit bonus limit (i.e. below −2), return false and leave the credits unchanged.

(iii) Otherwise subtract the requested credits, record a ticket code, and return true.

8) Add a new JUnit test method **emptyWallet()** to check that the entire positive balance of a wallet with no bonus allowance can successfully be used. The test should pass only if the initial credits are zero or above and the remaining credits after use are zero.

9) Add a new method named **getLastTicketCode()** to the EventTicketWallet class. This method must return the most recently issued ticket code. If no ticket has been issued, the method must return null. The method must not modify the state of the object.

10) Add a new JUnit test method named **lastTicketCodeIsReported()** to verify the correct behaviour of the getLastTicketCode() method. The test must confirm that getLastTicketCode() returns null before any successful useCredits() call, returns a non-null value after a successful useCredits() call, and remains unchanged after a failed useCredits() call.

11) Run the Event class and verify that the program output matches the expected output shown in the provided screenshot. Submit a screenshot of the console output. The screenshot must clearly display the complete output in a single image.

12) After all required modifications have been completed, run the EventTicketWalletTest class and verify that the test output matches the expected output shown in the provided screenshot. Submit a screenshot of the test execution. The screenshot must clearly display the complete output in a single image.

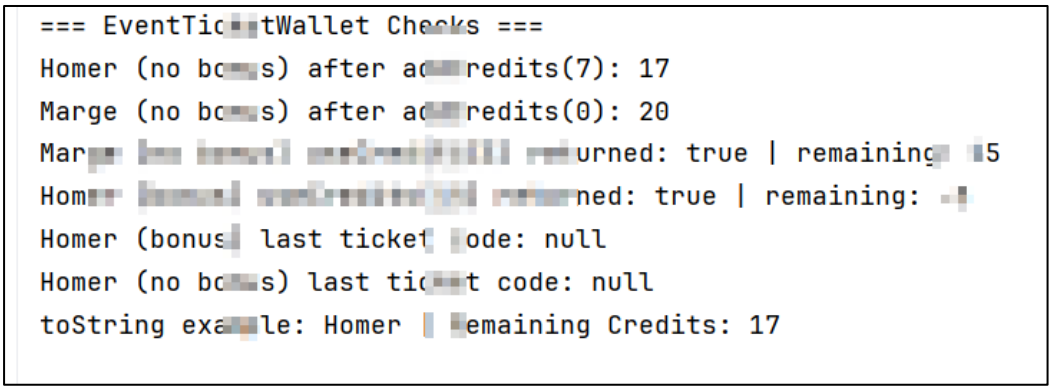
Marking Scheme

Marks will be awarded for each of the tasks according to the following scheme.

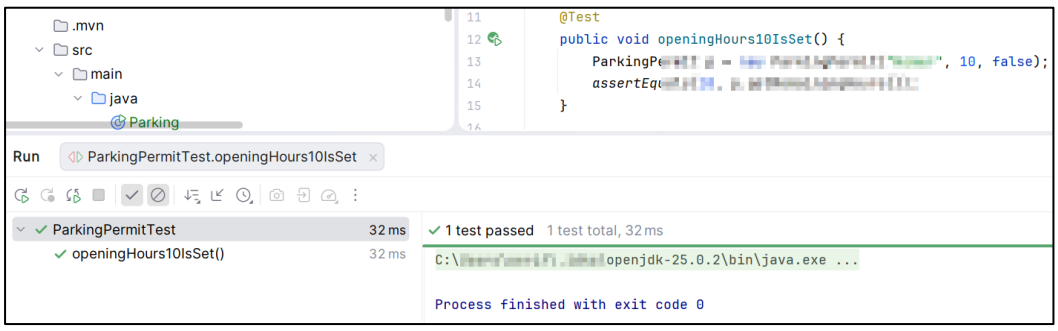
Task	1	2	3	4	5	6	7	8	9	10	11	12
Marks	4	2	2	4	4	4	4	4	4	4	2	2

Target Output

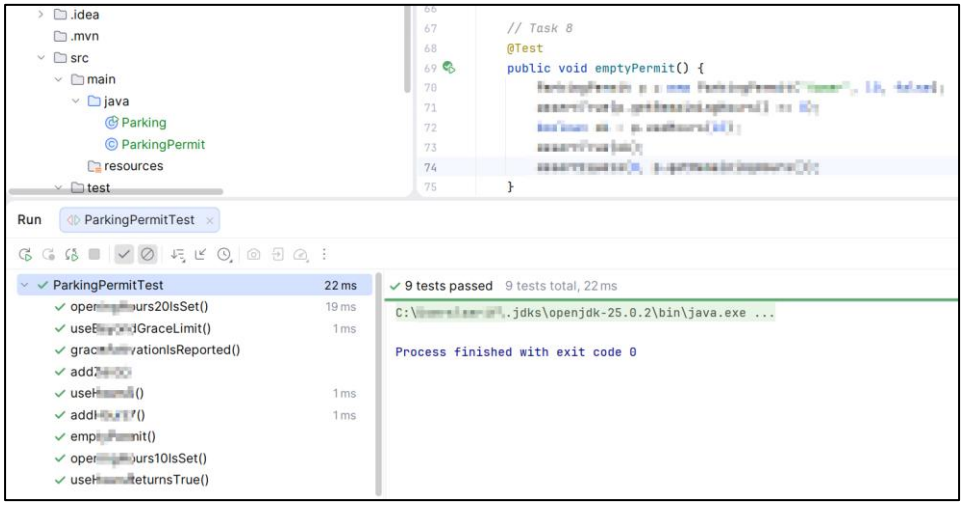
- Runing the **Event class** should output as the following screenshot.



- **After each JUnit test** completed, there should be an output as the following screenshot.



- **After all the modifications** are successfully made the **EventTicketWalletTest class** should output as the following screenshot.



Submission

This assessment is submitted **task by task** using the individual submission windows provided in Blackboard for each exam question.

For **each task**, students **must submit both the relevant code and a screenshot** showing the result of running that code. Submissions must be uploaded **only** to the Blackboard window corresponding to the relevant task.

For tasks involving **JUnit tests**, a screenshot of the test run is **mandatory**, regardless of whether the test passes or fails. The screenshot is required to verify that the code was executed. **If no screenshot is submitted for a JUnit task, the submission for that task will be awarded zero marks**, even if code is provided.

The following additional submissions are required:

- A screenshot showing the output from running the **Event** class after all tasks have been completed must be submitted to the designated Blackboard window for the Parking class execution.
- A screenshot showing the output from running the **EventTicketWalletTest** class after all tasks have been completed must be submitted to the designated Blackboard window for the JUnit test execution.

All screenshots must clearly show the full output. Students should ensure that the output panel is expanded so that the entire result is visible within a single screenshot.

Submissions uploaded to the incorrect Blackboard window may not be marked.

Feedback

Feedback on each submission will be provided in written form for each of the tasks identified in the specification. All results and feedback will be returned withing 20 working days of submission in accordance with University policy.

Plagiarism Declaration

By making a submission, you will be deemed to have made the following declaration of ownership. Source: https://www.ulster.ac.uk/learningenhancement/cqe/assessment-codeof-practice/coursework-submission.

"I declare that this is all my own work. Any material I have referred to has been accurately referenced and any contribution of Artificial Intelligence technology has been fully acknowledged. I understand the importance of academic integrity and have read and understood the University's General Regulation: Student Academic Integrity and the Academic Misconduct Procedure. I understand that I must not upload my work before, during or after submission to any unapproved plagiarism detectors or answer sharing platforms, or equivalent, and that only University-approved platforms should be used."